Отчёт по лабораторной работе

Разработка приложений на основе готовых библиотек машинного обучения

Тема: автоматическая предмодерация объявлений

1. Цель и условия работы

Цель работы — разобраться, какие прикладные задачи сегодня решаются готовыми моделями машинного обучения, и построить работающее решение на пяти типах контента: тексте, аудио, изображениях, видео и локальной языковой модели.

Работа велась в следующих условиях. Модели используются только предобученные: обучение и дообучение не проводились, в коде нет ни одной функции потерь, оптимизатора или обучающего цикла. Всё выполняется локально, без обращения к облачным провайдерам. Задействовано четыре разных фреймворка. Один из модулей заранее выбран кандидатом на последующую разработку API — им стал модуль обработки текста, поскольку он не требует файлов на входе и отвечает за миллисекунды.

Весь стек открытый: библиотеки распространяются под свободными лицензиями, веса моделей взяты из открытых репозиториев, языковая модель развёрнута через Ollama на собственном оборудовании. Платные подписки и облачные API не использовались.

1.1. Выбор прикладной области

Задание допускает как четыре несвязанные задачи, так и общую прикладную область. Я выбрал второе — предмодерацию объявлений на площадке частных объявлений, — потому что это даёт осмысленную систему метрик. В изолированной задаче можно сказать только «точность 0,9»; в прикладной области то же число превращается в «столько-то процентов потока уходит модератору, столько-то нарушений просачивается в публикацию». Второе проверяемо и обсуждаемо, первое — нет.

На связность исполнения это не влияет: модули остаются независимыми и запускаются по отдельности. Общими у них являются только справочник категорий и модуль замеров.

1.2. Оборудование и окружение

Компонент	Значение
Оборудование	Apple M4 Pro, 24 ГБ оперативной памяти
Операционная система	macOS 26.6
Язык и окружение	Python 3.12, изолированное виртуальное окружение
Ускоритель	встроенный GPU через Metal (MPS); часть модулей сознательно работает на процессоре — обоснование в разделах 5.1 и 5.3

Все числа в отчёте получены прогонами скриптов из репозитория на этой конфигурации. Исходные машинные результаты сохранены в каталоге *report/metrics/*, и скрипт сборки

отчёта читает измеренные значения прямо из этих файлов: после повторного прогона оценки отчёт пересобирается одной командой, без ручного переписывания таблиц.

2. Прикладная задача

Площадка частных объявлений. Пользователь публикует объявление из четырёх типов контента: текстовое описание, фотографии, видеообзор и голосовой комментарий. Объём потока не позволяет проверять каждое объявление вручную. Каждый модуль относит проверяемый контент к одной из трёх корзин: **allow** — публиковать, **review** — модератору, **reject** — отклонить.

2.1. Цена ошибки определяет всю систему метрик

Ошибки несимметричны. Ложная блокировка честного объявления обходится дороже пропущенного нарушения: продавец теряет деньги и уходит к конкуренту, тогда как пропуск обнаруживается жалобами и постмодерацией. Отсюда жёсткое ограничение — **точность на классе reject не ниже 0,95**, — а полнота максимизируется уже под ним. Поэтому корзин три: всё, в чём модель не уверена, передаётся человеку, а не блокируется.

Это же правило определило архитектуру: итоговый вердикт вычисляет детерминированное правило в коде, а языковая модель только формулирует объяснение. Решение о блокировке — не то место, где уместна вероятностная генерация.

3. Архитектура и обоснование стека

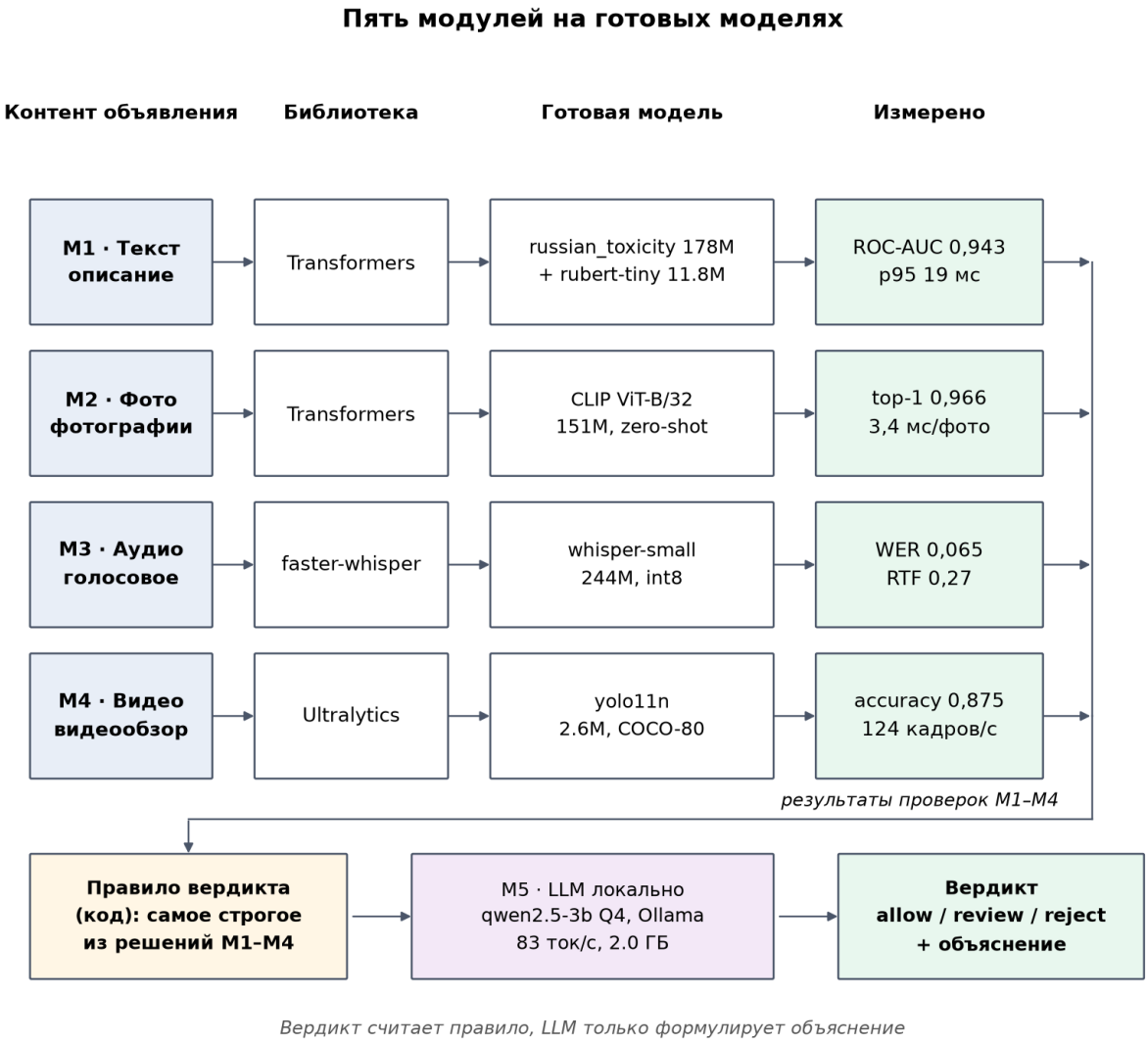


Рисунок 1 — Пять модулей, используемые библиотеки, готовые модели и измеренные показатели

3.1. Архитектурный стиль

Выбран стиль независимых модулей с общими справочными данными, а не единого конвейера: по условию это пять самостоятельных задач, отказ одного модуля не должен останавливать остальные, а стоимость запуска различается от миллисекунды на текст до секунд на видео — связывать такое в синхронный конвейер значило бы равнять всё по самому медленному.

Общим сделан только справочник категорий *catalog.json* — это данные, а не исполнение. До его введения у модулей были собственные списки категорий, противоречившие друг другу.

3.2. Обоснование выбора библиотек

Модуль	Библиотека	Почему именно она
M1, M2	Hugging Face Transformers	Препроцессинг поставляется вместе с весами: токенизатор и нормализация те же, что при

Модуль	Библиотека	Почему именно она
		обучении. Ручная реализация расходится в мелочах, и потеря качества остаётся незаметной. Библиотека также сообщает устройство модели через config, что позволило написать модуль, работающий и с multilabel-, и с бинарными классификаторами без правок
M3	faster-whisper (CTranslate2)	Квантизация int8 из коробки: модель small работает на процессоре в несколько раз быстрее реального времени. Встроенная фильтрация тишины ускоряет обработку и снижает число галлюцинаций
M4	Ultralytics	Берёт на себя чтение видео, масштабирование, подавление дублирующих рамок и отрисовку. Пропуск кадров выполняется на уровне загрузчика — пропущенные кадры не декодируются вовсе
M5	Ollama (llama.cpp)	Установка одной командой, использование Metal и, главное, структурированный вывод: передача JSON-схемы ограничивает генерацию так, что невалидный ответ невозможен

4. Ход работы и зоны ответственности

Этап	Содержание
1	Постановка задачи и ТЗ: выбрана прикладная область, определена цена ошибки, из неё выведены критерии приёмки и метрики
2	Окружение: Python 3.12 и ffmpeg через Homebrew, изолированное виртуальное окружение, глобальный интерпретатор не затронут
3	Модуль текста: инференс, подбор порогов, оценка, HTTP-обёртка. Выбран кандидатом на API — не требует файлов на входе
4-7	Модули изображений, аудио, видео и языковой модели: по каждому скрипт инференса и скрипт оценки с сохранением метрик
8	Унификация справочника категорий, сравнение моделей разного размера, замеры на процессоре и ускорителе
9	Расширение тестового набора для текста с 60 до 200 примеров с разбивкой по видам нарушений — этот шаг изменил вывод по модулю
10	Проверка результатов: кросс-валидация порогов текста, пересчёт метрики подмены категории для фото по рабочей логике решений, автоматическая

Этап	Содержание
	проверка текста объяснений LLM
11	Отчётность: все замеры сохраняются машиночитаемо, отчёт собирается из них скриптом

Единый модуль *common/bench.py* измеряет латентность, память и устройство одинаково во всех модулях — иначе цифры разных модулей несопоставимы. Результаты сохраняются в *report/metrics/*.json*, отчёт собирается из них, а не переписывается руками.

Работа выполнена индивидуально; распределение ролей между участниками не производилось.

5. Модули: реализация и результаты

По каждому модулю приводятся пять обязательных разделов: преимущества библиотеки, принцип работы модели и причины выбора, структура датасета, метрики качества, особенности реализации и эффективность. Сравнения моделей разного размера вынесены в раздел 6, сводка по критериям приёмки — в раздел 7.

5.1. M1 — текст: детекция нарушений в описании

Преимущества библиотеки

Transformers сводит задачу к трём операциям: загрузка весов по идентификатору, токенизация тем же токенизатором, которым модель обучалась, получение логитов. Дополнительно библиотека сообщает устройство модели — состав классов и тип задачи. Модуль читает эти поля и работает с моделями разного устройства без правок кода; без этого сравнить две модели было бы невозможно.

Принцип работы модели и причины выбора

Используются две модели в каскаде, и к этому привёл замер.

- Решение принимает *s-nlp/russian_toxicity_classifier* — энкодер на базе rubert-base, 178 млн параметров, бинарная голова. Текст разбивается на токены, слои self-attention набирают контекст, итоговое представление идёт в линейный слой.
- Вид нарушения добавляет *cointegrated/rubert-tiny-toxicity* — 11,8 млн параметров, 45 МБ, пять независимых сигмоид: отсутствие нарушений, оскорбление, нецензурная лексика, угроза, опасный контент.

Изначально взята была только лёгкая модель. На публичном корпусе она давала ROC-AUC 0,998 и выглядела достаточной. На собственном наборе объявлений выяснилось, что она детектирует только прямые оскорбления с бранными существительными, а угрозы и презрение пропускает почти полностью. Тяжёлая модель эти виды ловит, но бинарная и не сообщает, что именно нарушено, — а это требуется для формулировки причины отказа. Отсюда каскад: решение по тяжёлой модели, вид нарушения по лёгкой и только для уже помеченных текстов. На чистом потоке, а это около 95 % объявлений, вторая модель не запускается вовсе.

Структура датасета

Обучение: модели взяты готовыми, обучались на открытых корпусах русских токсичных комментариев. Для оценки используются два набора: публичный корпус (6000 текстов, 17,2 % нарушений) даёт устойчивые метрики, но модели обучались на данных того же происхождения, поэтому результат на нём — оценка сверху. Собственный набор объявлений (200 текстов) модели не видели.

Собственный набор строился не как «чистые против грубых», а по подгруппам, и именно это сделало его информативным:

Подгруппа	n	Содержание
plain	55	обычные нейтральные объявления
firm	30	законные резкие формулировки:

Подгруппа	n	Содержание
		«перекупщиков не беспокоить»
negative	15	резкие оценки товара, но не людей: «состояние ужасное»
insult	40	прямые оскорбления покупателей
contempt	25	презрение и дискриминация, без бранных слов
threat	15	угрозы, сформулированные без брани
borderline	20	грубость без оскорбительных существительных

Подгруппы firm и negative — проверка на ложные срабатывания: такие объявления публикуются ежедневно и блокировать их нельзя. Подгруппы contempt, threat и borderline — проверка на покрытие.

Метрики качества: покрытие видов нарушений

Первый вопрос к модели — видит ли она вообще тот или иной вид нарушения. Ответ на него не должен зависеть от выбранного порога, поэтому сравнение ведётся по медианной оценке внутри подгруппы: это характерное число, которое модель выдаёт на таких текстах.

Подгруппа	n	rubert-tiny (11,8M)	russian_toxicity (178M)
insult — прямые оскорбления	40	0,5671	**0,9542**
borderline — грубость без брани	20	0,0006	**0,0535**
threat — угрозы	15	0,0005	**0,0119**
contempt — презрение	25	0,0003	**0,0060**
plain — обычные объявления	55	0,0002	0,0011
firm — резкие, но законные	30	0,0002	0,0018
negative — негатив о товаре	15	0,0005	0,0031

Лёгкая модель отделяет от чистых текстов только прямые оскорбления: её медиана на них 0,5671 против 0,0002 на обычных объявлениях — разрыв в три порядка. На угрозах, презрении и грубости без брани медианы 0,0003–0,0006, то есть ровно того же порядка, что у чистых текстов. **Никакой порог не отделит одно от другого**, потому что модель их не различает.

Это видно и напрямую. На угрозах её специализированная голова threat выдаёт ноль:

```
«Если не придёшь на встречу — найду и проблемы будут, понял?»
   non-toxic=1.000  insult=0.000  obscenity=0.000  threat=0.000
```

Причина в обучающих данных: форумные корпуса токсичности содержат прямую брань, а деловая угроза в объявлении опасна смыслом, а не словами. Наличие метки threat в конфигурации модели ничего не гарантирует.

У тяжёлой модели разрыв между нарушениями и чистыми текстами сохраняется на всех подгруппах: 0,0535 и 0,0119 против 0,0011–0,0031. Разрыв невелик, но он есть, и порог его использует. Именно из-за того, что он невелик, рабочие пороги получаются очень низкими — это главное слабое место модуля, о нём ниже.

Модель	Публичный корпус	Объявления	p50 на текст
rubert-tiny, 11,8M	ROC-AUC 0,9977	0,8148	**1,09 мс**
russian_toxicity, 178M	ROC-AUC 0,9985	**0,9433**	12,79 мс
каскад	—	—	13,3 мс (p95 19,3)

ROC-AUC измеряет качество ранжирования и от выбора порога не зависит. На публичном корпусе модели неразличимы, на домене объявлений разрыв 13 пунктов.

Выбор порогов

Модель выдаёт не решение, а оценку от нуля до единицы. Решение получается сравнением этой оценки с порогами, и порог — это наш выбор, а не свойство модели. Корзин три, значит порогов два: ниже нижнего объявление публикуется, выше верхнего отклоняется, между ними уходит модератору. Ниже описано, как эти два числа получены; все три панели рисунка 2 относятся к этому выбору.

Шаг 1. Что требуется. Из цены ошибки (раздел 2.1) следуют два ограничения: точность на классе reject не ниже 0,95 и нагрузка на модераторов не выше 20 % потока. Полнота максимизируется при соблюдении обоих.

Шаг 2. Поправка на базовую частоту. Точность нельзя брать прямо с размеченной выборки: в ней 50 % нарушений, а в потоке ожидается около 5 %. Точность от этой доли зависит, полнота и доля ложных срабатываний — нет, поскольку последние две считаются внутри своего класса. Поэтому точность пересчитывается аналитически:

```
precision(p) = p · TPR / ( p · TPR + (1 − p) · FPR )
```

где p — ожидаемая доля нарушений, TPR — доля пойманных нарушений, FPR — доля ошибочно помеченных чистых объявлений. Физически пересэмплировать набор было бы расточительно: чтобы из 100 на 100 получить 5 %, пришлось бы выбросить 95 нарушений из ста.

Эффект показан на средней панели рисунка 2. Синяя кривая — точность, посчитанная прямо по выборке; зелёная — она же в пересчёте на поток с 5 % нарушений. При одной и той же полноте зелёная проходит ниже: в реальном потоке те же ложные срабатывания весят больше, потому что чистых объявлений значительно больше. Выбирать порог по синей кривой значило бы получить в эксплуатации точность ниже заявленной.

Шаг 3. Верхний порог. Берётся минимальное значение, при котором зелёная кривая достигает требуемых 0,95; среди подходящих выбирается то, что даёт наибольшую полноту. Получается 0,1372 — красная точка на средней панели: полнота 0,55 при

точности 1,00. На левой панели эта же точка лежит в левом нижнем углу ROC-кривой: доля ложных срабатываний на этой выборке равна нулю.

Шаг 4. Нижний порог. Он отвечает на другой вопрос: какую долю нарушений система обязана хотя бы показать человеку. Требование «не менее 95 %» даёт порог 0,0019, но тогда модератору уходит 37,5 % потока — больше допустимых 20 %. Перебор целевого значения показывает цену размена:

Цель по полноте	Порог review	Нагрузка на модераторов	Утечка в allow	Ложные отклонения
0,95	0,0019	37,5 % — сверх лимита	4,0 %	0 %
0,85	**0,0050**	**19,5 %**	**15,0 %**	**0 %**
0,75	0,0080	13,0 %	23,0 %	0 %
0,65	0,0255	6,5 %	33,0 %	0 %

Выбрана строка 0,85: нагрузка 19,5 % укладывается в ограничение. Это осознанный размен по правилу цены ошибки — жертвуем полнотой, а не точностью, потому что пропуск нарушения дешевле ложной блокировки. Итоговая пара порогов: 0,0050 и 0,1372.

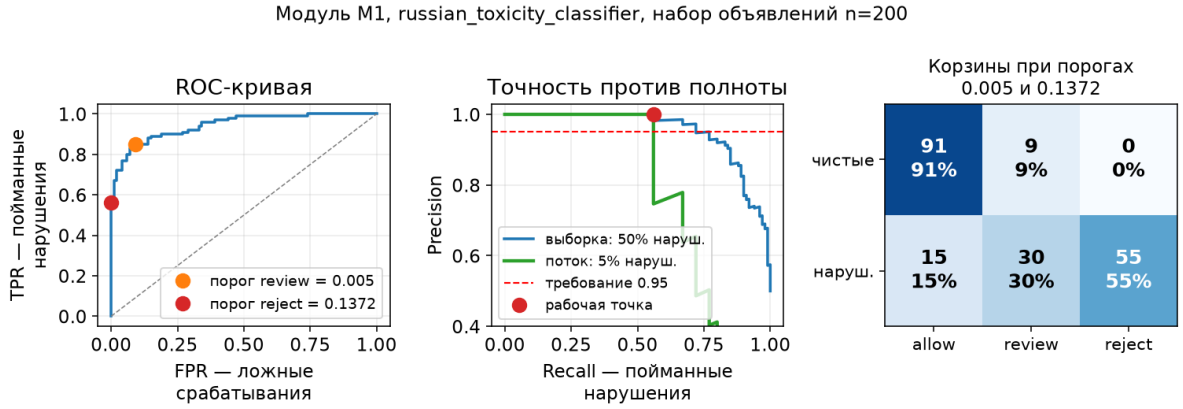


Рисунок 2 — Выбор порогов. Слева: ROC-кривая с отмеченными рабочими точками обоих порогов. В центре: точность против полноты на выборке и в пересчёте на реальную долю нарушений. Справа: распределение объявлений по трём корзинам при выбранных порогах

Рабочая точка

Правая панель рисунка 2 показывает итог — как делится поток. Обычная матрица ошибок здесь не подходит: она бинарна и схлопывает корзину ручной проверки, которой в системе отведена отдельная роль.

	allow — публикуем	review — модератору	reject — отклоняем
Чистые объявления (100)	**91**	9	**0**
Нарушения (100)	15	30	55

На этой выборке ни одно из 100 честных объявлений не отклонено. 15 нарушений из 100 прошли в публикацию: это и есть плата за выбор точки 0,85.

В разрезе видов нарушений картина такая:

Подгруппа	n	reject	review	allow
insult — прямые оскорбления	40	40	0	0
borderline — грубость без брани	20	7	10	3
threat — угрозы	15	5	8	2
contempt — презрение	25	3	12	10
чистые объявления	100	**0**	9	91

Прямые оскорбления отклоняются все. Угрозы и грубость без брани в основном попадают к модератору, а не отклоняются автоматически: оценки модели на них невысоки, и автоматический отказ по таким оценкам означал бы риск для честных объявлений. Хуже всего с презрением и дискриминацией: 10 случаев из 25 уходят в публикацию. Пропускаются и отдельные угрозы (2 из 15) — в том числе приведённая выше фраза «найду и проблемы будут»: её оценка тяжёлой моделью ниже порога review.

Проверка порогов на отложенных данных

Пороги выше подобраны на тех же 200 текстах, на которых посчитаны метрики, поэтому эти метрики оптимистичны: порог подстраивается под конкретные примеры. Чтобы оценить, как пороги поведут себя на новых объявлениях, проведена 5-кратная кросс-валидация: набор делится на 5 частей с сохранением долей подгрупп, пороги подбираются на 4 частях тем же алгоритмом, а решение по каждому тексту принимают пороги, которые этот текст не видели.

Показатель	Подбор и проверка на одних данных	Кросс-валидация, 5 частей
precision(reject) на выборке 50/50	1,000	0,967
precision(reject) в потоке с 5 % нарушений	1,000	**0,608**
Полнота reject	0,55	0,59
Нагрузка на модераторов	19,5 %	16,0 %
Утечка нарушений в allow	15,0 %	17,0 %
Честных объявлений отклонено	0 из 100	**2 из 100**
Разброс порога reject по частям	—	0,0195–0,1384

Кросс-валидация меняет вывод. На отложенных данных 2 честных объявления из 100 отклоняются, и в пересчёте на поток с 5 % нарушений точность отказа падает до 0,61 — **требование 0,95 на новых данных не подтверждается**. Причина видна в последней строке: порог reject между частями меняется в несколько раз, то есть ста чистых примеров недостаточно, чтобы надёжно поставить порог в области, где модель выдаёт оценки порядка сотых. Нижний порог (0,0050) ещё чувствительнее: сдвиг пунктуации меняет оценку на тысячные, и вердикт переходит из review в allow — такой случай найден в модуле аудио (раздел 5.3).

Вывод для эксплуатации: автоматический отказ по этим порогам допустим только после калибровки на размеченных объявлениях реального потока объёмом в тысячи примеров. До этого безопаснее направлять в review всё, что выше нижнего порога.

Продуктовые и инфраструктурные метрики

Показатель	Значение
Доля автоматических решений	80,5 %
Передано модераторам	19,5 %
Честных объявлений ошибочно отклонено	0 из 100 (кросс-валидация: 2 из 100)
Латентность каскада, p50 / p95	**13,3 / 19,3 мс** (CPU)
Латентность HTTP-сервиса, p50 / p95	15,6 / 17,8 мс
Пропускная способность, один поток	49 текстов/с
Пиковая оперативная память	918 МБ

Критерий по латентности (p95 менее 100 мс) выполнен с запасом: p95 каскада 19,3 мс, сервиса вместе с разбором HTTP — 17,8 мс.

Особенности реализации и эффективность

- **Каскад с ленивой загрузкой в CLI:** модель-объяснитель создаётся при первом нарушении, на чистом потоке не загружается вообще. В HTTP-сервисе, наоборот, обе модели грузятся и прогреваются при старте: ленивая загрузка давала 1,8 с на первом запросе с нарушением.
- **Система не выдумывает причину отказа:** вид нарушения называется только при уверенности выше 0,5, иначе объявление помечается как требующее ручной проверки. Без этого порога выбор максимума всегда что-нибудь возвращает — у лёгкой модели голова «опасный контент» срабатывает даже на текстах, которые она считает чистыми.
- **Работа на процессоре при доступном ускорителе:** для текста в 20-40 токенов перенос тензоров на GPU дороже вычислений.
- **Пороги отделены от кода** и несут пометку, на каком датасете и какой моделью подобраны, плюс предупреждение о непереносимости между доменами.
- **HTTP-обёртка** на FastAPI: эндпоинты /health, /predict и /predict/batch, валидация входа по длине текста. Старт с загрузкой и прогревом обеих моделей — 9,4 с, далее 15,6 мс на запрос (p50) вместе с разбором HTTP.

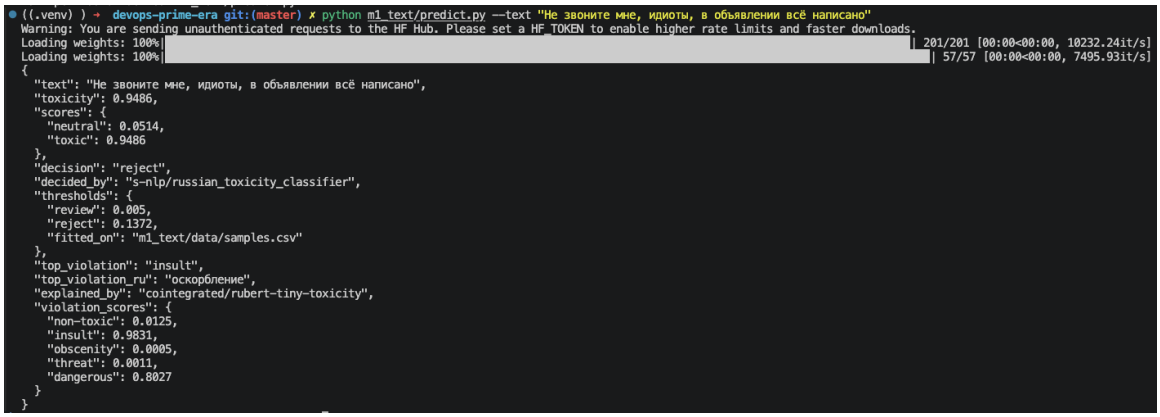


Рисунок 3 — Запуск M1 из командной строки: текст с оскорблением отклонён, вид нарушения определён второй ступенью каскада

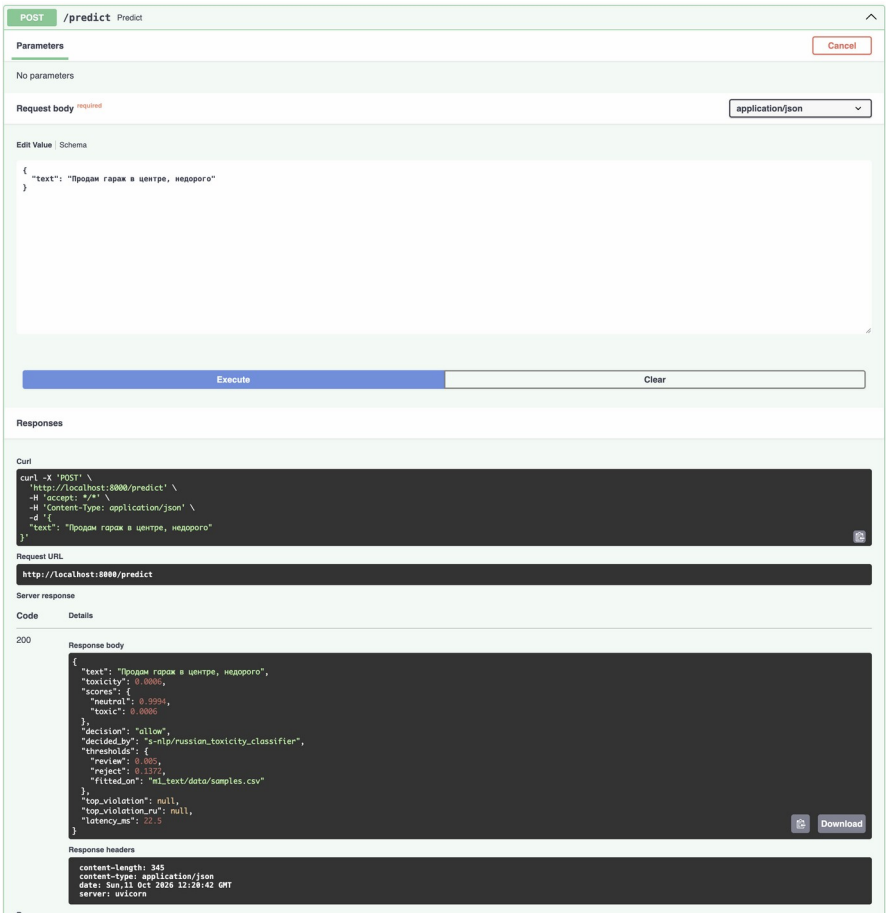


Рисунок 4 — HTTP-сервис M1: интерактивная документация FastAPI и ответ на запрос POST /predict

5.2. M2 — изображение: соответствие фото категории

Преимущества библиотеки

Та же Transformers, но здесь важно другое свойство: процессор модели содержит ровно тот препроцессинг, с которым она обучалась — размер, кадрирование, нормализацию по каналам. Ошибка в любом из этих чисел даёт молчаливую деградацию: модель работает, точность просто ниже.

Принцип работы модели и причины выбора

CLIP ViT-B/32 (151 млн параметров, 577 МБ) — две башни: Vision Transformer для изображений и трансформер для текста, обе проецируют вход в общее пространство на 512 измерений. Обучение шло на парах «картинка — подпись»: эмбеддинги верной пары сближались, неверной расходились. Следствие: чтобы классифицировать изображение, достаточно описать классы словами.

Причина выбора следует из задачи: категорий на площадке сотни, они добавляются и переименовываются, обучать классификатор под каждую нельзя. Новая категория — несколько строк в справочнике, а не цикл переобучения.

Структура датасета

Обучение: около 400 млн пар «изображение — подпись» из интернета. Инференс: файл JPEG или PNG плюс заявленная категория из справочника на 23 позиции. Оценка: Imagenette, валидационный сплит, 500 изображений, 10 классов с готовой разметкой;

классы интерпретируются как товарные категории. Для измерения детекции подмены каждое изображение проверяется дважды: с верной категорией (честное объявление) и с заведомо неверной (подмена). В справочник включены три «дистрактора» — скриншот, документ, селфи: типовые непригодные вложения, которые модель должна распознавать, а не натягивать на товарную категорию.

Метрики качества

Модель	Вариантов выбора	top-1	top-3
CLIP zero-shot	23 категории	0,970	0,998
CLIP zero-shot	23 + 3 дистрактора	**0,966**	0,998
ResNet50 обученный	10 классов	**0,998**	—

Цена универсальности измерена: 3,2 процентного пункта. Сравнение поставлено в пользу обученной модели — ResNet50 знает ровно эти 10 классов и его выход из 1000 сужен до нужных, а CLIP выбирает из 26 вариантов, 16 из которых в тестовом наборе вообще не встречаются. Добавление трёх дистракторов стоило CLIP 0,4 пункта: чем больше вариантов в справочнике, тем больше поводов ошибиться.

Основной сценарий модуля — решение по заявленной категории. Оно принимается той же функцией, что и в рабочем коде: allow, если распознана заявленная категория с уверенностью не ниже 0,5; review, если модель ни в чём не уверена; reject, если модель уверенно видит другую категорию.

	allow	review	reject
Честные объявления	**96,0 %**	1,6 %	2,4 %
Подменённая категория	0,2 %	1,6 %	**98,2 %**

Подмена не проходит в автоматическую публикацию в 99,8 % случаев, из них в 98,2 % она отклоняется автоматически. Точность отказа — 0,976 при равной доле честных и подменённых объявлений; честных фотографий ошибочно отклонено 2,4 %. Это часть случаев, где модель неверно распознала класс (ошибка top-1 — 3,4 %); остальные неуверенные распознавания порог отправил в review, а не в отказ.

Оговорка: неверная категория назначается сдвигом по справочнику, и чаще всего она визуально далека от истинной (собака против бензопилы). Подмена на близкую категорию — «Автомобили» вместо «Спецтехники» — может ловиться заметно хуже; такой набор в работе не собирался, и приведённые цифры относятся только к грубой подмене.

Размер батча	GPU (MPS), мс/изобр.	CPU, мс/изобр.	Выигрыш GPU
1	8,98	18,69	2,08×
4	4,97	8,37	1,68×
16	3,62	5,62	1,55×
32	3,41	5,12	1,50×

Батчинг даёт больше, чем GPU: переход к батчу из 32 ускоряет в 2,6 раза на GPU и в 3,7 раза на процессоре, тогда как переход с процессора на GPU — в 1,5–2,1 раза. Если поток фотографий позволяет накапливать пачки, это выгоднее, чем требовать ускоритель.

Особенности реализации и эффективность

- **Эмбеддинги категорий считаются один раз при инициализации:** тексты категорий не меняются от запроса к запросу, поэтому на запрос приходится только кодирование изображения и одно матричное умножение.
- **Промпт-ансамблирование:** на категорию три формулировки, их нормированные эмбеддинги усредняются — снижается зависимость от одной неудачной формулировки.
- **Температура берётся из модели,** а не подбирается: она обучена вместе с весами.
- **Одна функция решения для инференса и оценки:** метрика подмены считается тем же кодом, что принимает решение в эксплуатации, поэтому не может разойтись с ним.

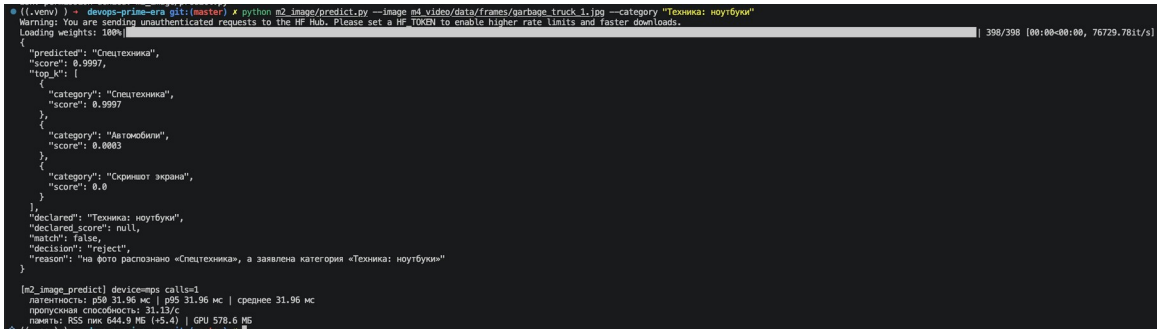


Рисунок 5 — Запуск M2: на фото мусоровоз, заявлена категория «Техника: ноутбуки» — объявление отклонено с причиной

5.3. M3 — аудио: распознавание речи

Преимущества библиотеки

faster-whisper — реализация Whisper на движке CTranslate2 вместо PyTorch с квантизацией int8 из коробки. Существенное ограничение платформы: CTranslate2 не поддерживает Metal, модель работает на процессоре. На использованном оборудовании это не проблема — в int8 отношение времени обработки к длительности записи (RTF) составляет 0,27. Ограничение зафиксировано явно, потому что при переносе на машину с CUDA картина изменится.

Принцип работы модели и причины выбора

Whisper small (244 млн параметров, 461 МБ в int8) — трансформер кодировщик-декодировщик. Аудио приводится к 16 кГц моно и превращается в лог-мел-спектрограмму, то есть по сути в картинку «частота на время». Кодировщик сворачивает её в последовательность представлений, декодировщик авторегрессивно порождает текст, попутно предсказывая язык, наличие речи и временные метки. Обучение шло на 680 тысячах часов разноязычного аудио, из-за чего модель устойчива к акцентам и шуму без настройки.

Модуль существует не ради красивой расшифровки, а чтобы текст попал в модуль M1: нарушения в голосовых иначе не отлавливаются вообще.

Структура датасета

Размеченных публичных корпусов под домен «голосовые комментарии к объявлениям» нет, поэтому набор синтезирован: 15 фраз на языке объявлений озвучены системным голосом macOS, эталонный текст известен по построению. Для приближения к реальным условиям добавлен белый шум двух уровней, итого 45 записей. Фактическое отношение сигнал/шум измерено по готовым файлам: около 13 дБ для умеренного шума и 7 дБ для сильного. Ограничение: синтезированная речь чище живой, полученные значения ошибки распознавания — нижняя граница.

Метрики качества

Условие	WER	CER	RTF	Согласованность вердиктов
чистая запись	0,065	0,016	0,267	93,3 %
умеренный шум, 13 дБ	0,194	0,048	0,264	93,3 %
сильный шум, 7 дБ	0,406	0,135	0,267	80,0 %

Главная метрика здесь не ошибка распознавания, а согласованность вердиктов — доля записей, где решение М1 по расшифровке совпало с решением по эталонному тексту. Сравнение первых двух строк показательно: при умеренном шуме ошибка распознавания выросла в 3,0 раза, а согласованность не изменилась — модель путает окончания, но оскорбление остаётся оскорблением. При сильном шуме согласованность падает до 80,0 %: опасны не любые ошибки, а те, что уничтожают ключевое слово:

```
эталон:      Не звоните мне идиоты, в объявлении всё написано русским языком
распознано:  Незвонительный геотез, в объявлении в санокисном русском языком.
вердикт:     reject → allow
```

Оскорбление растворилось в шуме, и нарушение прошло бы в публикацию. Именно это и обнаруживает выбранная метрика — показатель WER такого не показывает.

Единственное расхождение на чистой записи другого рода. Фраза «Все перекупщики тупые бараны, продаю только в нормальные руки» распознана почти дословно, с ошибками только в окончаниях, но вердикт сменился с review на allow. Оценка М1 для этого текста находится вблизи нижнего порога, и небольшого изменения формулировки хватает, чтобы перейти через него. Это следствие низких порогов М1 (раздел 5.1), а не ошибка распознавания.

Особенности реализации и эффективность

- **Квантизация int8** уменьшает размер весов и ускоряет вычисления на процессоре; сравнение с float32 в работе не проводилось, выбор сделан по рекомендации разработчиков faster-whisper.
- **Фильтр речевой активности** отсекает тишину до подачи в модель.
- **Функция распознавания возвращает генератор,** и вычисления происходят только при обходе. В коде это сделано явно — иначе замер времени показал бы микросекунды вместо реальной работы.

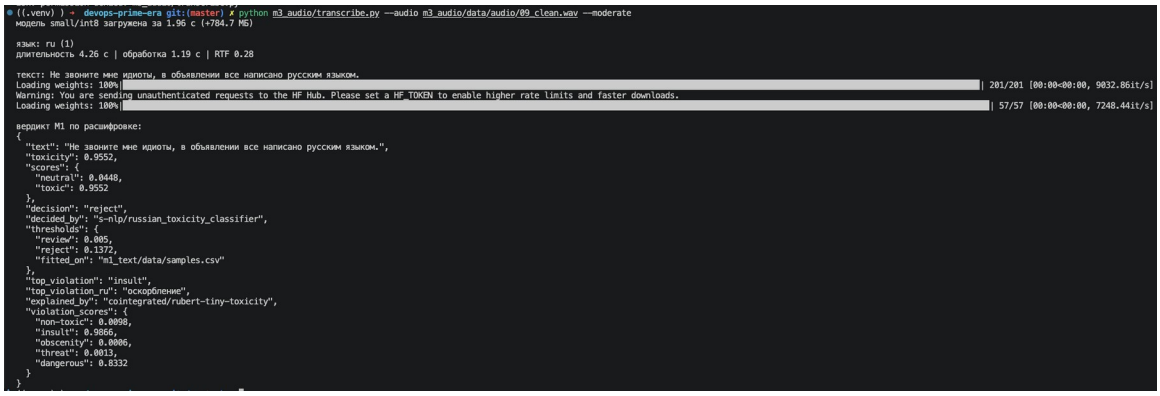


Рисунок 6 — Запуск M3: распознавание голосового комментария и передача расшифровки в M1

5.4. M4 — видео: проверка видеообзора товара

Преимущества библиотеки

Ultralytics берёт на себя чтение видео, масштабирование с сохранением пропорций, подавление дублирующих рамок и отрисовку. Параметр пропуска кадров реализован на уровне загрузчика — пропущенные кадры не декодируются вовсе. Потоковый режим отдаёт генератор результатов: видео обрабатывается покадрово и не грузится в память целиком.

Принцип работы модели и причины выбора

YOLO11n (2,6 млн параметров, 5,4 МБ) — одностадийный детектор без якорных рамок. Изображение проходит через свёрточный экстрактор признаков, признаки с разных масштабов объединяются, а голова напрямую предсказывает для каждой точки сетки смещения рамки, уверенность и класс. В отличие от двухстадийных архитектур нет отдельного этапа генерации кандидатов — отсюда скорость. 80 классов COCO хорошо ложатся на товарные категории: велосипед, автомобиль, диван, ноутбук, телевизор, холодильник, собака, кошка.

Структура датасета

Обучение: COCO, около 118 тысяч изображений, 80 классов, разметка прямоугольниками. Инженерное следствие: из 23 категорий справочника видеомодуль способен проверить 16, остальные помечены как непроверяемые. Оценка: 12 роликов по 6 секунд, собранных из фотографий Imagenette медленным наездом с панорамированием — для детектора это близко к тому, что снимает продавец, обходя товар. На каждый ролик две проверки: с верной категорией и с подменённой, итого 24. Ограничения: в таких роликах нет смазов и ухода объекта из кадра, а подменная категория назначается сдвигом по справочнику и визуально далека от истинной, поэтому её обнаружение — простая часть задачи.

Метрики качества

Шаг между анализируемыми кадрами выбран по данным, а не на глаз: при шаге 5 качество то же, что при полном переборе, а времени тратится в 4,7 раза меньше.

Шаг кадров	Accuracy	Подмены пойманы	Время (GPU), с
1	0,875	1,000	27,0

Шаг кадров	Accuracy	Подмены пойманы	Время (GPU), с
5	**0,875**	**1,000**	**5,8**
10	0,833	1,000	3,3
15	0,833	1,000	2,6

Время указано за полный проход по 24 проверкам.

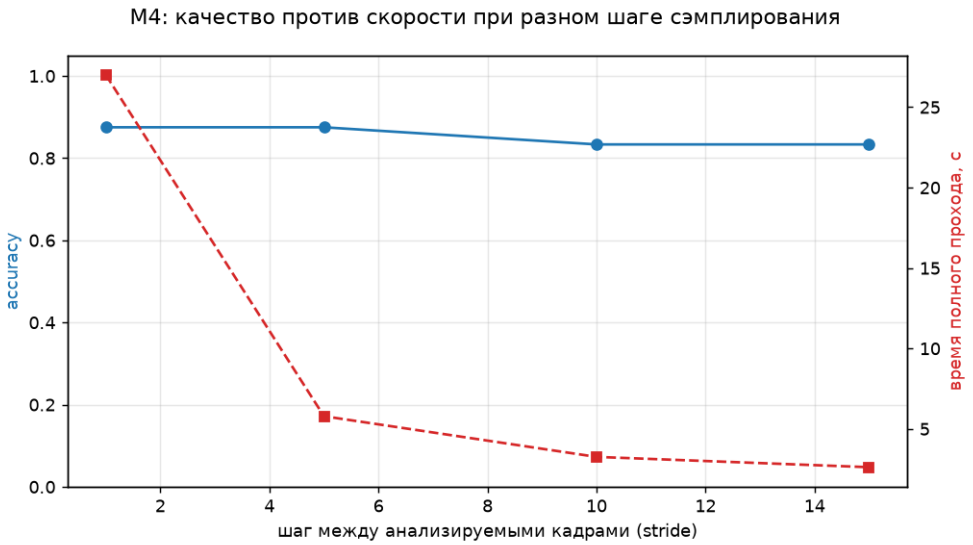


Рисунок 7 — Зависимость качества и времени обработки от шага между анализируемыми кадрами

Категория	Роликов	Подтверждено
Животные: собаки	4	4 / 4
Спецтехника	4	4 / 4
Спорт: мячи	4	1 / 4

Ошибки сосредоточены в категории «Спорт: мячи», и причина конкретна: класс COCO «спортивный мяч» обучен на футбольных, теннисных и баскетбольных мячах, а мяч для гольфа крупным планом детектор принимает за «яблоко» и другие округлые предметы. Вывод для эксплуатации: в каталог допустимо класть только категории, надёжно покрытые классами обучающего набора.

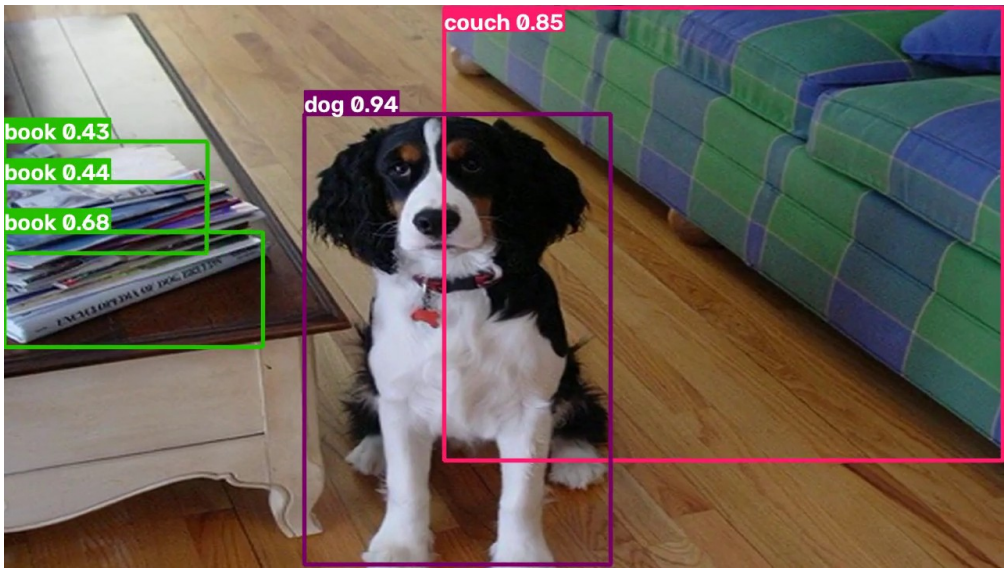


Рисунок 8 — Кадр из выходного видео M4 с рамками детектора: ролик категории «Животные: собаки»

Особенности реализации и эффективность

- **Фильтр единичных срабатываний:** объект засчитывается только если найден минимум в трёх кадрах — одиночное срабатывание на одном кадре почти наверняка ошибка детектора. Отсеянные классы возвращаются отдельным полем, чтобы решение было объяснимо.
- **Категория подтверждается набором классов, а не одним.** При этом «Спецтехника» отделена от «Автомобилей»: пока они были слиты, мусоровоз подтверждался как легковой автомобиль, и подмены ловились в 91,7 % вместо нынешних 100 %.
- **Скорость:** при шаге 5 ролик длительностью 30 секунд обрабатывается примерно за 1,2 с на GPU и 2,6 с на процессоре (пересчёт с 6-секундных роликов).

```
((.venv) ) ➜  devops-prime-era git:(master) ✗ python3 m4_video/detect_video.py --video m4_video/data/video/English_springer_1.mp4 --category "Животные: собаки"
{
  "video": "English_springer_1.mp4",
  "frames_analyzed": 30,
  "elapsed_s": 1.06,
  "fps": 28.3,
  "device": "mps",
  "stride": 5,
  "objects": {
    "book": {
      "frames": 30,
      "share_of_frames": 1.0,
      "best_confidence": 0.77
    },
    "dog": {
      "frames": 30,
      "share_of_frames": 1.0,
      "best_confidence": 0.956
    },
    "couch": {
      "frames": 30,
      "share_of_frames": 1.0,
      "best_confidence": 0.889
    }
  },
  "rejected_as_noise": {},
  "memory_mb": {
    "rss_delta": 331.9,
    "gpu_allocated": 10.0
  },
  "declared": "Животные: собаки",
  "expected_classes": [
    "dog"
  ],
  "found_classes": [
    "dog"
  ],
  "declared_present": true,
  "decision": "allow",
  "reason": null
}
```

Рисунок 9 — Запуск M4: найденные объекты и подтверждение заявленной категории

5.5. M5 — локальная языковая модель: вердикт и объяснение

Преимущества библиотеки

Ollama — обёртка над llama.cpp, снимающая возню с квантизацией и выгрузкой слоёв на ускоритель: модель ставится одной командой и поднимает HTTP API на локальном адресе. Решающая возможность — структурированный вывод: в запрос передаётся JSON-схема, и сервер ограничивает генерацию так, что невалидный JSON получиться не может. Это не постобработка ответа, а ограничение на этапе выбора токенов — отсюда 100 % разбираемых ответов. Сервер также отдаёт собственные счётчики токенов, поэтому скорость берётся от него, а не оценивается клиентом.

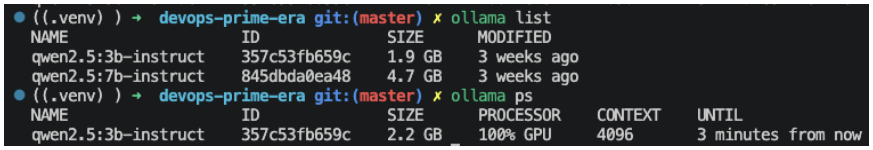


Рисунок 10 — Локальное развёртывание: модели qwen2.5 установлены, 3B загружена и работает на GPU

Принцип работы модели и причины выбора

Qwen2.5-Instruct в квантизации Q4_K_M — трансформер-декодировщик, порождающий текст по одному токену с обусловливанием на всё предыдущее. Версия Instruct дообучена следовать инструкциям, поэтому держит формат ответа и системный промпт. Квантизация означает хранение весов примерно в 4 битах на параметр вместо 16: модель на 3 млрд параметров умещается в 1,9 ГБ.

Принципиальное ограничение роли: языковая модель **не принимает решение**. Вердикт вычисляет детерминированное правило — «побеждает самое строгое решение среди модулей». Модель получает готовые числовые результаты M1-M4 и делает то, что правилом не выражается: формулирует объяснение для продавца и сводку для модератора.

Структура датасета

Обучение: многоязычный веб-корпус порядка 18 трлн токенов плюс этап настройки на инструкциях. Инференс: агрегированный JSON с решениями модулей M1-M4; контент повторно не подаётся. Оценка: 16 синтетических кейсов, построенных систематическим перебором сочетаний решений — всё чисто, по одному нарушению в каждом модуле, конфликтующие сочетания, неполные объявления. Эталонный вердикт вычисляется тем же правилом, что и в эксплуатации, а не проставляется руками.

Метрики качества

Каждая модель прогнана по 16 кейсам 3 раза — 48 запросов, чтобы отделить устойчивый результат от случайности сэмплирования. Помимо вердикта, текст для продавца проверяется автоматически на формальные дефекты: служебные термины системы (allow, review, reject), слова из смешанных алфавитов, пустой или слишком длинный ответ.

Показатель	qwen2.5-7b	qwen2.5-3b	Критерий ТЗ
Валидный JSON с первой попытки	100 %	100 %	не ниже 95 %
Совпадение вердикта с правилом	81,2 %	**100,0 %**	не ниже 90 %

Показатель	qwen2.5-7b	qwen2.5-3b	Критерий ТЗ
Текст для продавца без дефектов	**100,0 %**	93,8 %	—
Скорость генерации	44,4 ток/с	**83,4 ток/с**	не ниже 15 ток/с
Время ответа, среднее / p95	2,86 / 3,89 с	**1,09 / 1,45 с**	—
Память под модель	4521 МБ	**2055 МБ**	не выше 12 ГБ

Модель на 7 млрд параметров ошиблась с вердиктом 9 раз из 48, и все ошибки воспроизводятся во всех повторах на одних и тех же кейсах: «текст:reject + фото:review», «текст:review + фото:reject», «только текст, чисто». В смешанных кейсах она видит два разных решения и выбирает мягкое, хотя правило приоритета задано в системном промпте явно; в кейсе с одним текстом отсутствующие проверки она трактует как повод для ручной проверки. Совпадение вердикта у неё к тому же нестабильно между прогонами: при предыдущем прогоне того же набора было 87,5 %. Меньшая модель применяет правило буквально и не ошиблась ни разу.

Зато по тексту для продавца картина обратная: у 3B служебные термины найдены в 3 ответах из 48, например: «Было выявлено одно незавершенное рассмотрение (review) в отношении фотографии. Пожалуйста, уточните детали или предоставьте дополнительные фотографии для проверки.» У 7B таких ответов нет. Выбор 3B сделан потому, что ошибка в тексте исправима постобработкой (замена служебных терминов), а ошибка смысла — нет; но это размен, а не превосходство по всем показателям.

Это подтверждает архитектурное решение: если бы вердикт выносила языковая модель, выбор размера стал бы критичным — неверный статус получало бы заметное число объявлений, систематически в сторону смягчения. Поскольку вердикт считает правило, ошибка модели на решение не влияет.

Особенности реализации и эффективность

- **Схема вместо просьбы:** формат задаётся JSON-схемой в параметре запроса, а не фразой «ответь в формате JSON». Отсюда 100 % валидности.
- **Объяснение не противоречит вердикту:** если вердикт модели разошёлся с правилом, объяснение запрашивается повторно с уже вычисленным вердиктом в промпте. Без этого продавец прочитал бы «объявление на проверке» при фактическом отказе.
- **Низкая температура (0,2):** задача — не творчество, а воспроизводимая формулировка.
- **Память меряется через служебный эндпоинт сервера,** а перед замером все модели выгружаются. Две итерации до правильного замера описаны в разделе 8.

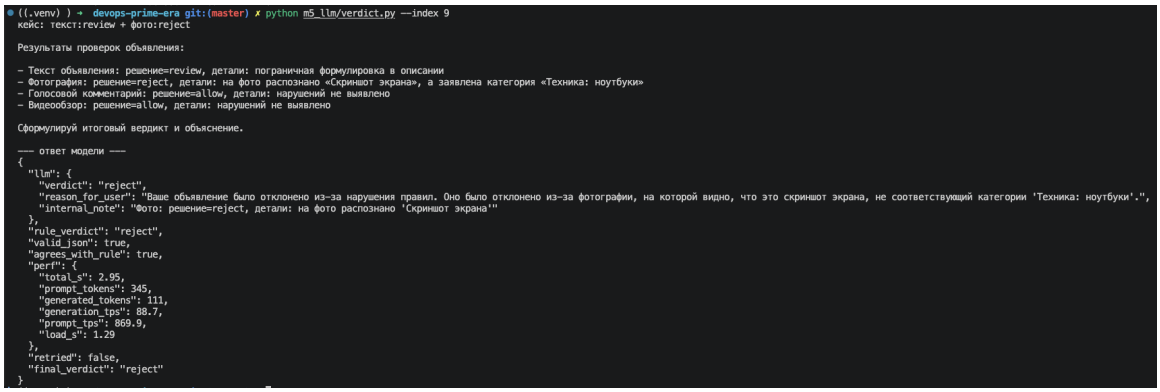


Рисунок 11 — Запуск M5: результаты проверок M1–M4 на входе и JSON-ответ локальной модели

6. Сравнение моделей разного размера

В четырёх модулях замерялись модели разного размера, чтобы обоснование выбора опиралось на измерение, а не на цифры из карточки модели. В модуле изображений вместо этого сравнивались zero-shot и обученный классификатор (раздел 5.2).

Модуль	Модели	Качество	Скорость	Вывод
M1 текст	rubert-tiny 11,8M против russian_toxicity 178M	публичный корпус: 0,9977 против 0,9985 объявления: 0,8148 против **0,9433**	1,09 мс против 12,79 мс	**Окупается.** На исходном домене модели неразличимы при 12-кратной разнице в скорости, а на новом разрыв 13 пунктов: лёгкая не видит целых видов нарушений
M3 аудио	whisper tiny / base / small 39M / 74M / 244M	согласованность вердиктов: чисто — 93,3 % / 93,3 % / 93,3 % сильный шум — 73,3 % / 73,3 % / 80,0 %	RTF 0,059 / 0,097 / 0,267	**Не окупается на чистом звуке:** все три размера дают одинаковую согласованность при разной скорости. Окупается под шумом
M4 видео	yolo11n 2,6M против yolo11s 9,4M	при шаге 5: 0,875 против 0,875 при шаге 15: 0,833 против 0,875	5,8 с против 6,7 с	**Не окупается при рабочем шаге:** крупная тратит на 16 % больше времени при той же точности. Окупается при редком анализе кадров
M5 LLM	qwen2.5 7B против qwen2.5 3B	вердикт: 81,2 % против **100,0 %** текст без дефектов: **100,0 %** против 93,8 %	44,4 против **83,4** ток/с 4521 против **2055** МБ	**Не окупается для этой задачи:** меньшая модель точнее следует правилу и вдвое быстрее, крупная пишет чище. Задача требует буквального исполнения правила, дефекты текста исправимы постобработкой

Отдельно измерен выигрыш от ускорителя. В модуле изображений переход с процессора на GPU даёт 1,5–2,1 раза, а батчинг — 2,6–3,7 раза; в модуле видео GPU даёт 1,8–3,3 раза, причём выигрыш растёт с размером модели и падает с увеличением шага кадров.

Общий вывод: крупная модель однозначно окупилась только в модуле текста — там решала полнота покрытия видов нарушений, которой у лёгкой модели не было. В аудио и видео крупная модель выигрывает лишь в тяжёлых условиях (сильный шум, редкий анализ кадров), а в LLM меньшая модель лучше подходит под задачу. Сравнивать модели надо в условиях эксплуатации и на данных своего домена.

7. Выполнение критериев приёмки

Сводка по критериям из технического задания (SPEC.md). Для M1 приведены обе оценки — на данных подбора порогов и при кросс-валидации, — поскольку они расходятся.

Модуль	Критерий	Факт	Статус
M1	precision(reject) ≥ 0,95	1,00 на данных подбора; 0,61 при кросс-валидации в пересчёте на поток	**не подтверждён** на отложенных данных
M1	доля review ≤ 20 %	19,5 %; при кросс-валидации 16,0 %	выполнен
M1	p95 < 100 мс на CPU	19,3 мс	выполнен
M2	top-1 ≥ 0,85	0,966	выполнен
M2	recall подмены ≥ 0,80 при precision ≥ 0,90	0,998 при 0,976	выполнен
M2	< 200 мс на изображение	9,0 мс GPU, 18,7 мс CPU	выполнен
M3	WER ≤ 0,20 при нормальном качестве	0,065	выполнен
M3	согласованность вердиктов ≥ 0,90	чистая запись 93,3 % / умеренный шум 93,3 % / сильный шум 80,0 %	выполнен без шума и при умеренном шуме, **не выполнен** при сильном
M3	RTF ≤ 0,5	0,27	выполнен
M4	accuracy ≥ 0,85	0,875 при шаге 5	выполнен
M4	ролик 30 с за ≤ 30 с	≈ 1,2 с GPU, ≈ 2,6 с CPU	выполнен
M5	валидный JSON ≥ 95 %, вердикт ≥ 90 %, ≥ 15 ток/с, ≤ 12 ГБ	qwen2.5-3b: 100 %, 100 %, 83 ток/с, 2,0 ГБ	выполнен

Не выполнены два критерия. Согласованность вердиктов в M3 при сильном шуме упирается в качество распознавания; ошибка здесь ведёт к пропуску нарушения, а не к ложной блокировке. Точность отказа в M1 упирается в объём размеченных объявлений: на отложенных данных появились ложные отказы честных объявлений, поэтому автоматический отказ в M1 рекомендуется включать только после калибровки на реальном потоке.

8. Возникшие проблемы и их решения

Часть проблем — ошибки измерения, дававшие правдоподобные, но неверные цифры. Такие случаи опаснее явных сбоев: они не падают.

8.1. Инфраструктура и совместимость

Проблема	Решение
Загрузка весов с Hugging Face зависала на нуле байт: процесс жив, файл не растёт	Причина в транспортном бэкенде xet. Отключён переменной HF_HUB_DISABLE_XET=1, загрузка пошла по обычному HTTP
Библиотека datasets версии 4 перестала поддерживать датасеты со скриптами загрузки	Выбраны репозитории, хранящие данные в формате parquet; наличие parquet-файлов проверяется до скачивания
Хранилище fast.ai недоступно из сети	Тот же датасет взят из зеркала на Hugging Face
В Transformers версии 5 метод получения эмбеддингов возвращает объект, а не тензор	Нужное поле — pooler_output. Корректность проверена сверкой с выходом полного прохода модели: совпадение до седьмого знака
Загрузка второй языковой модели для сравнения зависла на 4,1 ГБ из 4,9	Вместо модели того же размера другого семейства взята модель меньшего размера того же семейства. Эксперимент стал полезнее: вопрос «окупается ли размер» информативнее вопроса «какой вендор лучше»
Оболочка zsh не разбивает переменную на аргументы, и цикл запуска прогонов молча ничего не выполнял	Команды запускаются явно, без подстановки строки аргументов
Тестовые данные для изображений и видео не попадали в репозиторий, и на чистой копии оценка не воспроизводилась	Добавлены скрипты загрузки Imagenette и подготовки кадров для роликов

8.2. Ошибки измерения

Проблема	Решение
Общий модуль замеров сообщал устройство «GPU» для модулей, фактически работающих на процессоре: брал доступное в системе, а не используемое	Фактическое устройство передаётся в замер явно; все затронутые измерения перепроведены
Память под языковую модель измерялась по процессу с именем ollama и показывала 41 МБ вместо 4,5 ГБ	Веса держит дочерний процесс llama-server, а не управляющий
После исправления замер стал суммировать все такие процессы и приписал модели на 1,9 ГБ семь гигабайт	Сервер держит модель в памяти ещё пять минут после запроса, поэтому при сравнении двух моделей в памяти были обе. Теперь перед замером все модели выгружаются, размер берётся из служебного эндпоинта
Функция распознавания речи возвращает генератор: замер времени показывал бы микросекунды вместо реальной работы	Результат явно материализуется внутри замеряемого участка
Пороги подбирались на выборке с долей нарушений 17 % и 50 %, тогда как в эксплуатации ожидается 5 %	Точность пересчитывается на ожидаемую долю аналитически через доли верных и ложных срабатываний — без выбрасывания размеченных данных
Пороги текста подбирались и проверялись на	Добавлена кросс-валидация; она показала, что

Проблема	Решение
одних и тех же 200 примерах, оценка была завышена	требование к точности на новых данных не подтверждается (раздел 5.1)
Метрика подмены категории для фото считала подмену пойманной при любом предсказании, кроме самой подменной категории, без порога уверенности	Метрика считается той же функцией решения, что работает в эксплуатации, с разбивкой по трём корзинам и точностью отказа
Уровни шума в аудио назывались по децибелам, но задавались амплитудой генератора	Фактическое отношение сигнал/шум измерено по готовым файлам

8.3. Содержательные проблемы

Проблема	Решение
У модулей фото и видео были собственные списки категорий, противоречившие друг другу	Введён единый справочник. Побочный эффект: стало видно, что видеомодуль способен проверить только 16 категорий из 23
Категории «Автомобили» и «Спецтехника» были слиты, мусоровоз подтверждался как легковой автомобиль	Категории разделены по классам детектора; обнаружение подмены выросло с 91,7 % до 100 %
Выбранная для текста модель оказалась слепа к угрозам и презрению. Обнаружено только после расширения тестового набора с 60 до 200 примеров с разбивкой по видам нарушений	Решение передано модели с лучшим покрытием, разбивка по видам нарушений оставлена за лёгкой моделью — каскад с ленивой загрузкой второй
Ленивая загрузка второй модели в HTTP-сервисе давала 1,8 с на первом запросе с нарушением	В сервисе обе модели загружаются и прогреваются при старте
Выбор максимума по видам нарушений всегда что-нибудь возвращал: голова «опасный контент» срабатывала даже на текстах, которые модель считает чистыми	Введён порог уверенности для объяснения. Ниже него система сообщает, что вид нарушения не определён, вместо того чтобы назвать выдуманную причину
Два требования ТЗ оказались несовместимы: «поймать 95 % нарушений» и «не более 20 % потока на ручную проверку»	Построена кривая размена, рабочая точка выбрана по цене ошибки
При расхождении вердикта LLM с правилом объяснение для продавца описывало бы не тот вердикт	Объяснение запрашивается повторно с вычисленным правилом вердиктом
Класс COCO «спортивный мяч» не покрывает мячи для гольфа	Ограничение зафиксировано в справочнике и в отчёте; категории без надёжного покрытия помечены как непроверяемые

9. Выводы

9.1. Что подтвердилось на практике

- **Готовые модели решают прикладные задачи без обучения.** Пять содержательных задач закрыты предобученными весами: от детектора на 5 МБ до языковой модели на 3 млрд параметров.
- **Тестовый набор определяет, что вы увидите.** Пока доменный набор для текста состоял из 60 примеров «обычные объявления против грубых», лёгкая модель показывала ROC-AUC 0,95 и выглядела достаточной. Как только добавились угрозы без брани, презрение и законные резкие формулировки, её показатель упал до 0,81, а у тяжёлой остался 0,94. Модели не изменились — изменился набор.
- **Оценка на данных подбора завышена.** Пороги текста, подобранные на 200 примерах, давали точность отказа 1,00; кросс-валидация показала 0,61 в пересчёте на поток. Для порогов в области сотых долей двухсот примеров мало.
- **Модель детектирует то, что видела, а не то, что названо в её метках.** У лёгкой модели есть голова «угроза», но на фразе «найду и проблемы будут» она выдаёт ноль. Наличие метки в конфигурации ничего не гарантирует.
- **Правильная метрика важнее хорошей модели.** В модуле аудио рост ошибки распознавания в 3,0 раза при умеренном шуме не снизил согласованность вердиктов, зато при сильном шуме отдельные ошибки меняли вердикт на противоположный. Измеряй мы только ошибку распознавания — этого не было бы видно.
- **Крупная модель окупается не всегда.** Однозначно окупилась только в модуле текста; в аудио и видео — лишь в тяжёлых условиях, в LLM меньшая модель лучше подошла под задачу (раздел 6).
- **Инженерная оптимизация может стоить дороже железа.** Батчинг ускоряет в 2,6–3,7 раза, а переход на GPU — в 1,5–2,1. Требовать ускоритель там, где достаточно накопить пачку запросов, — дорогая ошибка.
- **Требования могут оказаться несовместимыми, и это нормальный результат.** Правильный ответ — не подкручивать метрику, а предъявить кривую размена и выбрать точку по цене ошибки.

9.2. Ограничения работы

Что	В чём ограничение
Текст, публичный корпус	модели обучались на данных того же происхождения — оценка сверху
Текст, доменный набор	200 примеров, составлены автором: стилистика однороднее реальной
Текст, пороги	при кросс-валидации точность отказа 0,61 в пересчёте на поток вместо требуемых 0,95; нужна калибровка на реальных объявлениях
Изображения	подменная категория визуально далека от истинной; подмена на близкую категорию не измерялась
Аудио	синтезированная речь чище живой, ошибка занижена; int8 не сравнивался с float32
Видео	ролики собраны из фотографий трёх классов; 7

Что	В чём ограничение
	категорий из 23 не проверяются
Языковая модель	16 синтетических кейсов; качество текста проверялось только формальными правилами

9.3. Направления доработки

- Собрать доменный набор для текста из реальных объявлений с естественной долей нарушений и перекалибровать пороги.
- Записать живые голосовые сообщения и видеообзоры вместо синтезированных.
- Добавить в оценку фото и видео подмену на близкую категорию.
- Проверить whisper-medium и yolo11m: окупается ли рост качества замедлением.
- Сравнить Ollama с MLX — нативный для Apple Silicon рантайм.

10. Процесс разработки с ИИ-агентом

Работа выполнялась в режиме агентной разработки. Ниже описаны инструменты, модель, пайплайн и расход ресурсов.

10.1. Сетап

Компонент	Что использовалось
Агент	Claude Code, десктопное приложение
Модель	Claude Opus 5
Инструменты агента	чтение и правка файлов, запуск команд оболочки, фоновые задачи для долгих загрузок, панель браузера для проверки документов, скилл генерации .docx
Среда	macOS 26.6, Apple M4 Pro, Python 3.12 в изолированном окружении
Контроль версий	git, репозиторий создавался вручную

10.2. Расход ресурсов

Показатель	Значение
Размер контекста сессии	445 065 токенов, 45 % окна в 1 млн
Из них переписка	381 161 токен (86 %)
Определения инструментов	52 846 токенов
Расход тарифного плана	17 % пятичасового лимита, 3 % недельного (план Pro)

Важная оговорка: 445 тысяч токенов — это размер контекста на конец работы, а не суммарно оплаченные токены. Кумулятивный расход выше, поскольку контекст пересылается на каждом шаге; реальное потребление отражают проценты тарифного плана.

10.3. Пайплайн

Работа шла итерациями «сформулировать — реализовать — измерить — пересмотреть», а не одним проходом. Характерные шаги:

- **Постановка.** Первая версия плана начиналась с выбора моделей; это было отвергнуто и переделано в правильном порядке: прикладная задача, затем ТЗ с критериями, затем модели и только потом метрики.
- **Реализация модуля.** Сначала скрипт инференса с проверкой на одном примере, затем скрипт оценки с сохранением метрик в JSON. Долгие загрузки моделей и датасетов выносились в фоновые задачи, чтобы писать код параллельно.
- **Проверка.** Каждый модуль прогонялся на реальных данных сразу после написания. Это и выявляло проблемы из раздела 8 — например, расхождение API в новой версии библиотеки обнаружилось на первом же запуске.
- **Пересмотр.** Расширение тестового набора для текста изменило вывод по модулю и потребовало переписать его реализацию на каскад из двух моделей. Отдельное ревью

готовой работы нашло завышенную оценку порогов и упрощённую метрику подмены — после исправления выводы по M1 и M5 стали осторожнее. Агентная разработка даёт скорость, но проверять выводы всё равно приходится данными.

- **Отчётность.** Отчёт генерируется скриптом, который читает измеренные значения из сохранённых метрик, поэтому после перезапуска оценки он пересобирается без ручной правки.

10.4. Что оказалось трудным

Все технические трудности перечислены в разделе 8, и большинство из них — именно трудности агентной работы. Отдельно стоит выделить два класса:

- **Молчаливые ошибки измерения.** Замер памяти языковой модели дважды подряд выдавал правдоподобные, но неверные цифры — сначала в сто раз меньше, затем втрое больше. Ни один из вариантов не вызывал ошибку, и заметить это можно было только сверкой с ожидаемым порядком величины.
- **Смена API в свежих версиях библиотек.** Два случая: изменившийся тип возвращаемого значения и удалённая поддержка части датасетов. Оба решались чтением фактического поведения, а не документации.

11. Репозиторий

https://github.com/<ваш-аккаунт>/<имя-репозитория>

(ссылку подставить в REPO_URL в report/make_docx.py после публикации)

11.1. Состав

```
SPEC.md                техническое задание с критериями приёмки
README.md              обзор проекта и инструкции запуска
catalog.json           единый справочник категорий для M2 и M4
requirements.txt       прямые зависимости с зафиксированными версиями
requirements.lock.txt  полный слепок окружения (92 пакета)
common/bench.py        единый модуль замеров для всех модулей
m1_text/               текст:  predict / evaluate / api / bench_api
m2_image/              фото:   predict / evaluate / download_data
m3_audio/              аудио:  transcribe / evaluate / make_testset
m4_video/              видео:  detect_video / evaluate / make_testset
m5_llm/                LLM:    verdict / benchmark / make_cases
report/metrics/        машинные результаты всех прогонов (JSON)
report/img/            графики, схемы и скриншоты
report/docx/           этот отчёт (.docx и .pdf)
report/make_docx.py    сборка отчёта из измеренных метрик
```

11.2. Установка и запуск

```
brew install python@3.12 ffmpeg ollama
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt
ollama serve                      # в отдельном терминале
ollama pull qwen2.5:3b-instruct

# текст: инференс, метрики, HTTP-сервис
.venv/bin/python m1_text/predict.py --text "Продам гараж, документы готовы"
.venv/bin/python m1_text/evaluate.py --dataset m1_text/data/samples.csv \
```

```
    --prior 0.05 --target-recall 0.85 --cv 5 --tag domain_final
.venv/bin/uvicorn m1_text.api:app --port 8000

# данные для фото и видео
.venv/bin/python m2_image/download_data.py
.venv/bin/python m4_video/prepare_frames.py
.venv/bin/python m4_video/make_testset.py

# изображение, аудио, видео, языковая модель
.venv/bin/python m2_image/evaluate.py --limit 500 --perf-devices mps cpu
.venv/bin/python m3_audio/evaluate.py --sizes tiny base small
.venv/bin/python m4_video/evaluate.py --models yolo11n.pt yolo11s.pt \
    --devices mps cpu --strides 1 5 10 15
.venv/bin/python m5_llm/benchmark.py \
    --models qwen2.5:7b-instruct qwen2.5:3b-instruct --repeats 3

# отчёт
.venv/bin/python report/make_diagram.py
.venv/bin/python report/make_docx.py
```

11.3. Открытость стека

Компонент	Лицензия
Hugging Face Transformers, PyTorch, Ultralytics	Apache 2.0 / BSD / AGPL
faster-whisper, CTranslate2, Ollama, llama.cpp	MIT
Модели: rubert-tiny-toxicity, russian_toxicity_classifier, CLIP, Whisper, YOLO11, Qwen2.5	открытые веса
Датасеты: публичный корпус русских комментариев, Imagenette	открытые наборы данных

Платные подписки и облачные API не использовались, все вычисления выполняются локально.